

CPa’s effectiveness in identifying Faulty code changes:

In this experiment, we used *CPa* to identify the correct predictions made by the CL-FP models and analysed their effectiveness for this task. However, in practice, the objective of the developers is to identify faulty code changes without the need for exhaustive manual verification. We therefore extend our analysis to examine the effectiveness of *CPa* in identifying faulty code changes. For instance, let us consider once again the experimental configuration with CodeBERT4JIT on the QT dataset, in which *CPa* achieved the highest precision and recall. The QT dataset test set comprises 2571 instances, of which 2390 are clean and 181 are faulty. When applying CodeBERT4JIT, to make predictions for this test set, the model classifies 2478 instances as *Clean* and 93 as *Fault-prone*. Among these, 2326 are truly clean (true negatives) and 29 are truly faulty (true positives). Thus, CodeBERT4JIT correctly identifies 29 of the 181 faulty code changes in the dataset, achieving a fault identification rate¹ of $\frac{29}{181} = 0.16$. Applying *CPa* with 95% certainty results in 2269 predictions (23 *Fault-prone* and 2246 *Clean*) being flagged as potentially correct, representing 88% of the test set; and 302 predictions (12%) being flagged as uncertain. Out of the 2269 predictions flagged as potentially correct, 2150 are truly correct predictions (10 faulty and 2140 clean). Consequently, *CPa* at 95% certainty enables the identification of 10 of the 181 faulty code changes in the dataset, achieving a fault identification rate of $\frac{10}{181} = 0.06$. As mentioned above, *CPa* flags as potentially correct only a subset of predictions out of all the correct predictions made by the underlying CL-FP models. Hence, its ability to identify faulty code changes is inherently lower than that of the underlying predictive model. By adjusting the certainty level, practitioners can increase *CPa*’s accuracy. For instance, at 90% certainty *CPa* flags 2505 predictions (97% of the test set) as potentially correct, of which 2320 are correct (21 faulty and 2292 clean). In this setting, *CPa* enables the identification of 21 faulty code changes, achieving a fault identification rate of $\frac{21}{181} = 0.12$. Further decreasing the certainty level enables *CPa* to flag more predictions as potentially correct, thereby enabling the identification of more faulty code changes. However, it’s important to understand that *CPa*’s objective is not to replace the CL-FP model in better predicting faults, but to provide guidance on which predictions to trust. For example, although CodeBERT4JIT makes 2355 correct predictions on the QT test set, practitioners do not know in advance which subset of predictions are the correct ones. By relying on the 93 instances that CodeBERT4JIT predicted as *Fault-prone*, they would only recover 29 faulty code changes, with a fault identification accuracy² of $\frac{29}{93} = 0.31$. This means that by relying on CodeBERT4JIT’s predictions, practitioners would miss 152 (181-29) faulty code changes. To identify the remaining 152 faulty code changes, they would need to manually inspect the 2478 *Clean*

¹To prevent confusion with the results presented earlier, we do not use the term *recall* in this analysis. Specifically, the fault identification rate refers to the proportion of correctly predicted faulty code changes, out of all faulty code changes in the test set.

²To prevent confusion with the results presented earlier, we avoid using the term *precision*. In this analysis, fault identification accuracy represents the proportion of correctly predicted faulty code changes relative to all code changes that the CL-FP model classifies as *Fault-prone*.

predictions. We apply *CPa* on top of a CL-FP model to flag CL-FP predictions as potentially correct or uncertain. The underlying rationale is twofold:

1. Predictions flagged as potentially correct can be trusted, with the given certainty. Thereby, the *Fault-prone* predictions flagged as potentially correct should undergo manual verification to identify and localize faults.
2. Predictions flagged as uncertain should be treated with skepticism and should also be further investigated, provided sufficient QA resources are available.

Following this rationale, by relying on the 23 *Fault-prone* predictions made by CodeBERT4JIT that *CPa* flagged as potentially correct with 95% certainty, practitioners would recover 10 faulty changes with a fault identification accuracy $\frac{10}{23} = 0.44$. Next, by manually inspecting the 302 (12%) of CodeBERT4JIT’s predictions that *CPa* flagged as uncertain, practitioners would recover 65 additional faults. Altogether, this strategy enables the identification of 10+65 = 75 faults, yielding a total fault identification accuracy of $\frac{75}{181} = 0.4$. This means that by relying on *CPa* applied to CodeBERT4JIT’s predictions, practitioners would miss 106 (181-75) faulty code changes, instead of 152 as observed when using CodeBERT4JIT without *CPa*. Applying the same reasoning, we computed the fault identification accuracy for all experimental configurations. For this analysis, we define *fault identification accuracy*, denoted as *id accuracy*, as the number of identified faults over the number of investigated code changes. The detailed results of these computations can be found in our repository.

For the baseline, we assume that developers prioritize predictions in two stages:

- 1:** investigate all instances predicted as *Fault-prone* by the CL-FP model;
 - 2:** proceed to investigate the instances predicted as *Clean* by the CL-FP model
- In this setting, the column *#investigated* indicates the number of predictions (*Fault-prone* or *Clean*, depending on the priority level) that the developer must manually verify. The column *#faults identified* denotes the number of faulty code changes actually uncovered, based on the ground truth label.

For *CPa*, we adopt a similar strategy but aligned with its flagging mechanism:

Priority 1: investigate the *Fault-prone* predictions flagged as potentially correct by *CPa*;

Priority 2: proceed to investigate the predictions flagged as uncertain by *CPa*.

Here, the column *#investigated* reflects the number of *Fault-prone* predictions flagged as potentially correct by *CPa* that the developer must manually verify. The column *#faults identified* denotes the number of faulty code changes actually uncovered, based on the ground truth label. We then compute the identification rate as $\%identif. = \frac{\#faultsidentified}{\#changesinvestigated}$.